

Java - Les bases

Ecrire des programmes clairs,
etape par etape.



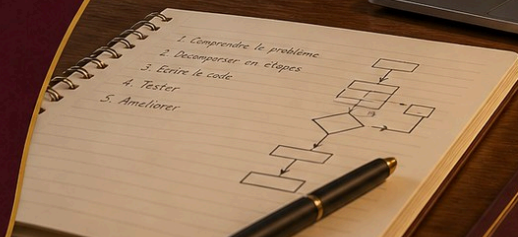
Variables —◆



Methodes —◆



Objets —◆



FORMATION JAVA - LES BASES

Java

Ecrire des programmes clairs, etape par
etape.

DanielCraft - Livre debutant - Pack Data/SQL

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

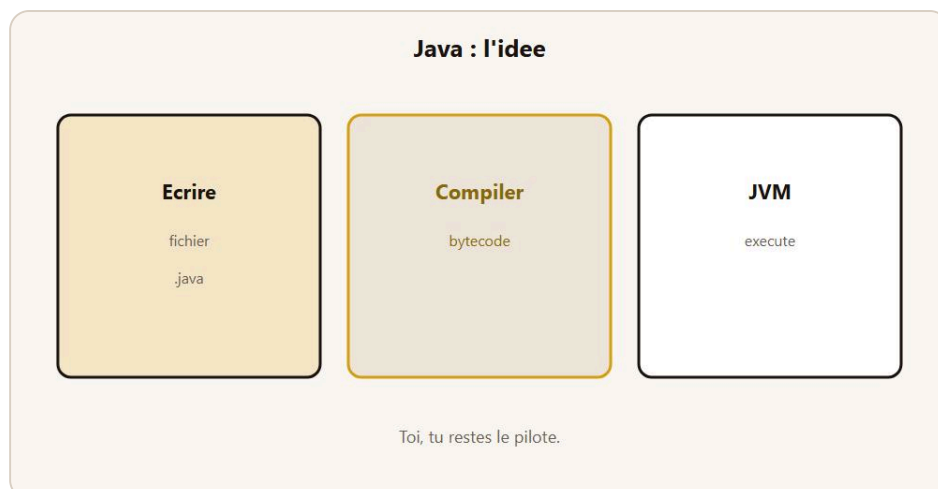
1. [Chapitre 1 - C'est quoi Java ?](#)
 - [Ce que ce n'est pas](#)
 - [Petite histoire](#)
 - [A toi](#)
2. [Chapitre 2 - Installer et lancer \(idée\)](#)
 - [A toi](#)
3. [Chapitre 3 - Premier programme](#)
 - [A toi](#)
4. [Chapitre 4 - Variables](#)
 - [A toi](#)
5. [Chapitre 5 - Types de base](#)
 - [A toi](#)
6. [Chapitre 6 - Conditions](#)
 - [A toi](#)
7. [Chapitre 7 - Boucles](#)
 - [A toi](#)
8. [Chapitre 8 - Methodes](#)
 - [A toi](#)
9. [Chapitre 9 - Tableaux](#)
 - [A toi](#)
10. [Chapitre 10 - Classes et objets](#)
 - [A toi](#)
11. [Chapitre 11 - Constructeurs](#)
 - [A toi](#)
12. [Chapitre 12 - Lire les erreurs](#)
 - [A toi](#)
13. [Chapitre 13 - Mini-projet](#)
 - [A toi](#)
14. [Chapitre 14 - Retenir l'essentiel](#)
15. [Chapitre 15 - Atelier variables](#)
16. [Chapitre 16 - Atelier methodes](#)
17. [Chapitre 17 - Atelier classe](#)
18. [Chapitre 18 - Erreurs classiques](#)
19. [Chapitre 19 - Bonnes pratiques](#)
20. [Quiz final](#)
 - [Question 1](#)

- [Question 2](#)
- [Question 3](#)
- [Question 4](#)
- [Question 5](#)
- [Question 6](#)
- [Question 7](#)
- [Question 8](#)
- [Question 9](#)
- [Question 10](#)
- [Question 11](#)
- [Question 12](#)
- [Corrige](#)

21. [Bravo](#)

- [Suite](#)

Chapitre 1 - C'est quoi Java ?



Java : tu écris du code, la machine virtuelle exécute.

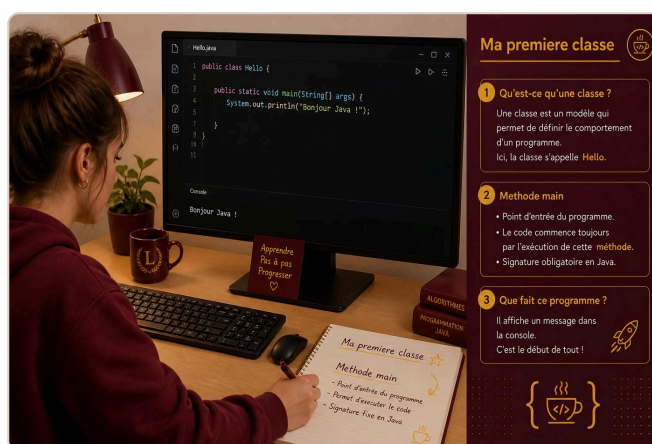
Java est un langage pour écrire des programmes qui tournent sur beaucoup de machines grâce à la **JVM** (machine virtuelle). Tu écris du `.java`, tu compiles en bytecode, la JVM exécute. Chez DanielCraft, on commence simple : variables, conditions, méthodes, puis objets. Lea veut comprendre avant Spring. Max a peur du jargon. Sam dit : "Java, c'est d'abord de la clarté."

★ A RETENIR

Java = code source + compilation + execution via la JVM. Toi, tu restes le pilote.

Ce que ce n'est pas

Ce n'est pas JavaScript. Ce n'est pas obligatoire d'apprendre Android le jour 1. Ce n'est pas "que des objets" avant de savoir compter. Ce n'est pas magie : une erreur de type se lit et se corrige.



Exemple : Lea ouvre sa première classe.

Petite histoire

Lea écrit un Hello World, sourit, puis casse volontairement une ligne pour lire l'erreur. Max confond Java et JavaScript une semaine. Sam affiche les deux logos et tranche.

A toi

Ecris pourquoi tu veux Java (job, etudes, curiosite).

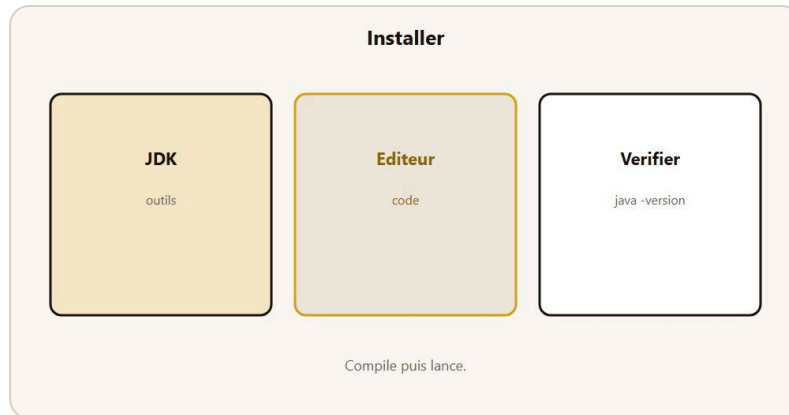
⚠ ATTENTION

Sans compiler / lancer, tu ne "sens" pas le cycle Java.

✦ ASTUCE

Garde un dossier `java-bases` pour tous les exercices.

Chapitre 2 - Installer et lancer (idee)



JDK + fichier .java : le duo de depart.

Tu as besoin d'un **JDK** (outils pour compiler et lancer). Puis un editeur (VS Code, IntelliJ Community...). Chez DanielCraft : verifier `java -version` et `javac -version`. Lea cree `Bonjour.java`. Max compile, puis execute.

Le cycle : ecrire -> `javac` -> `java NomDeLaClasse`. Les IDE font souvent ca en un bouton : comprends le geste quand meme.

★ A RETENIR

JDK + fichier .java + compile + lance.

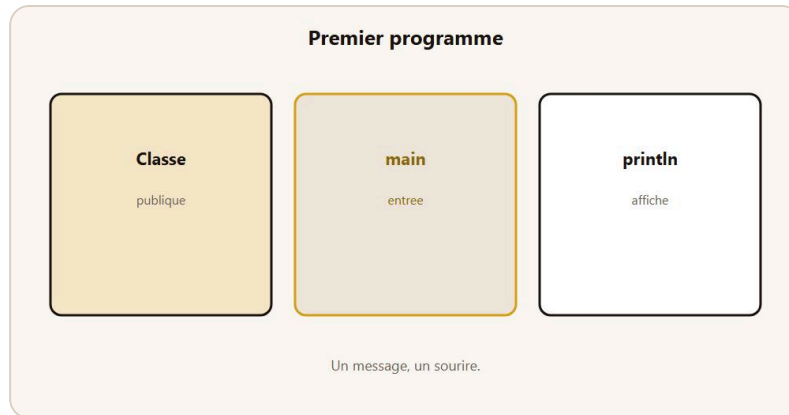
A toi

Note ta version Java affichee par `java -version`.

⚠ ATTENTION

Class public et nom de fichier doivent matcher (souvent).

Chapitre 3 - Premier programme



Classe + methode main : le point d'entree.

```
public class Bonjour {
    public static void main(String[] args) {
        System.out.println("Bonjour");
    }
}
```

`main` est le **point d'entree**. `println` affiche une ligne. Chez DanielCraft, Lea change le texte, recompile, relance. Max oublie les accolades : l'erreur le ramene.

★ A RETENIR

Classe + main + println = premier contact.

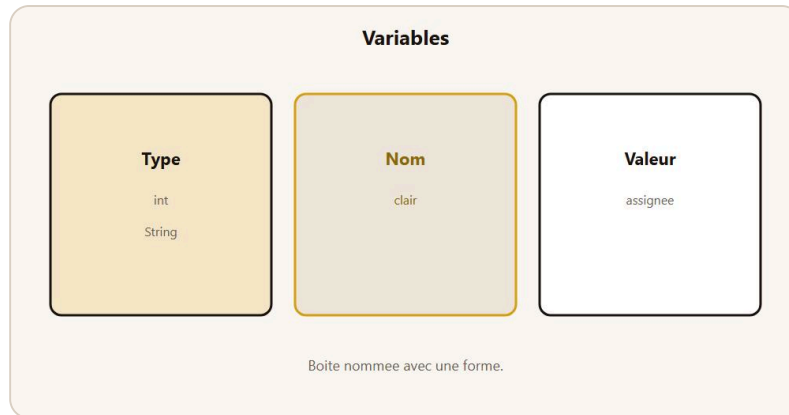
A toi

Change le message, ajoute une deuxieme ligne.

♦ ASTUCE

Une modification = un test. Pas dix d'un coup.

Chapitre 4 - Variables



Variable = boîte nommée avec une valeur.

Une **variable** range une valeur sous un nom. En Java, tu declares le type.

```
int age = 20;  
String prenom = "Lea";
```

★ A RETENIR

Type + nom + valeur. La boîte a une forme.

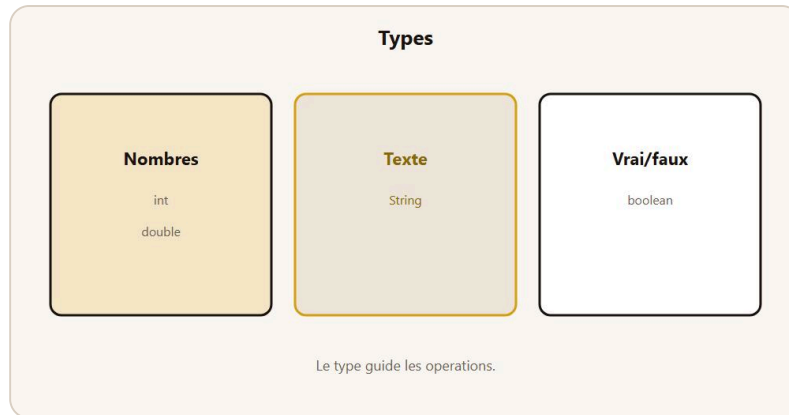
A toi

Declare `prix` (double) et `enStock` (boolean), affiche-les.

⚠ ATTENTION

Utiliser une variable avant de l'initialiser : souvent erreur.

Chapitre 5 - Types de base



`int`, `double`, `boolean`, `String` : les types utiles.

`int`, `long`, `double`, `boolean`, `char`, `String` (objet texte). Chez DanielCraft, on reste sur ces briques. Lea calcule un total `double`. Max compare des `boolean`. Sam rappelle : `String` n'est pas un type primitif, mais tu l'utilises partout.

★ A RETENIR

Choisir le type selon l'usage : compteur, montant, texte, vrai/faux.

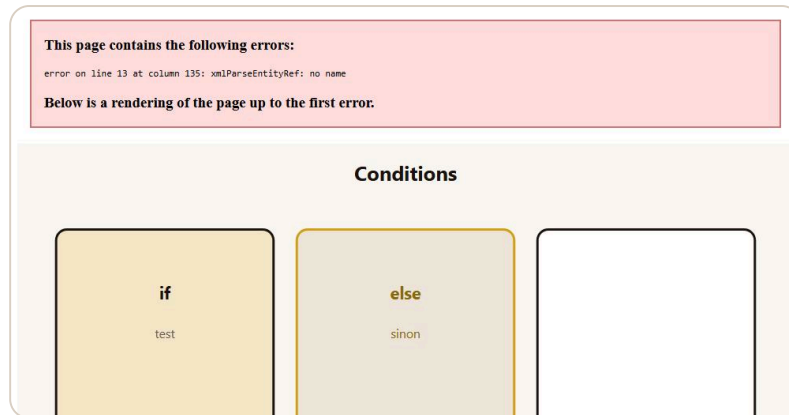
A toi

Pour un stock de produits : quels types pour quantite, prix, nom ?

♦ ASTUCE

Montants : `double` pour debuter ; plus tard tu verras `BigDecimal`.

Chapitre 6 - Conditions



if / else : choisir un chemin.

```
if (age >= 18) {
    System.out.println("Majeur");
} else {
    System.out.println("Mineur");
}
```

&&, ||, ! combinent. Chez DanielCraft, Lea écrit la condition en français d'abord.

★ A RETENIR

if / else = choisir un chemin selon une condition.

A toi

Si `note >= 10` affiche "OK", sinon "A revoir".

⚠ ATTENTION

`=` assigne, `==` compare (pour les primitifs).

Chapitre 7 - Boucles



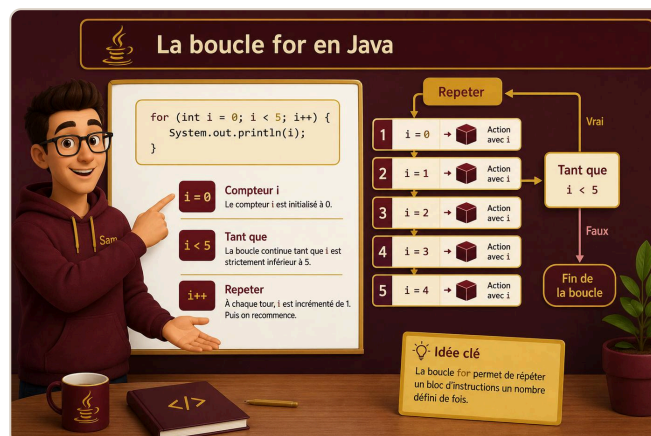
for / while : repeter sans copier-coller.

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

while repete tant que c'est vrai. Chez DanielCraft, Sam dessine le compteur. Max cree une boucle infinie une fois : Ctrl+C, lecon.

★ A RETENIR

Boucle = repeter. Surveille la condition d'arret.



Exemple : Sam compte avec une boucle.

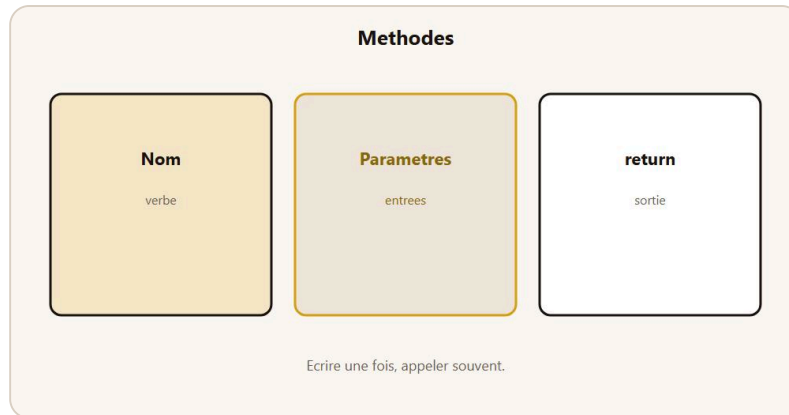
A toi

Affiche les nombres de 1 à 10.

⚠ ATTENTION

Oublier d'avancer le compteur dans while = boucle infinie.

Chapitre 8 - Methodes



Methode : ecrire une fois, appeler souvent.

Une **methode** regroupe des instructions. Tu l'appelles avec un nom.

```
static int doubleDe(int x) {
    return x * 2;
}
```

Chez DanielCraft, Lea decoupe "lire", "calculer", "afficher". Max copie-colle trois fois : Sam force une methode.

★ A RETENIR

Methode = ecrire une fois, appeler souvent.

A toi

Ecris `max(int a, int b)` qui renvoie le plus grand.

♦ ASTUCE

Noms de methodes = verbes (`calculerTotal`).

Chapitre 9 - Tableaux

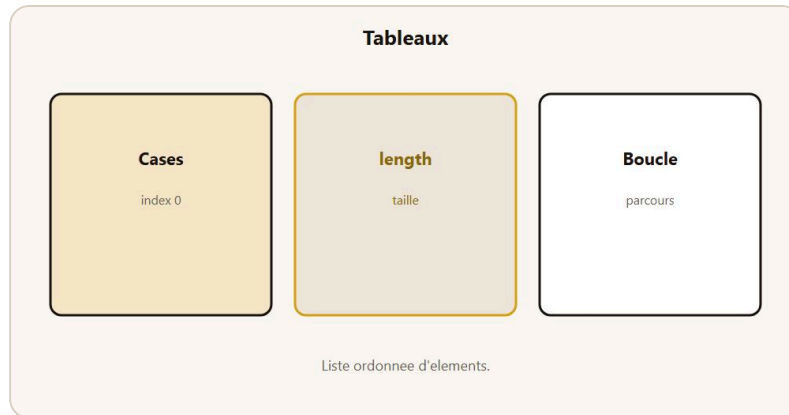


Tableau : une liste d'elements indexes.

```
int[] notes = {12, 15, 9};  
System.out.println(notes[0]);  
System.out.println(notes.length);
```

Index commence a **0**. Boucle **for** pour parcourir. Chez DanielCraft, Max deborde (`notes[3]`) : exception. Lea utilise `.length`.

★ A RETENIR

Tableau = cases numerotees a partir de 0.

A toi

Somme des elements d'un tableau de 4 entiers.

⚠ ATTENTION

Index hors limites = erreur a l'execution.

Chapitre 10 - Classes et objets



Classe = plan. Objet = exemplaire.

Une **classe** est un plan. Un **objet** est un exemplaire.

```
public class Client {  
    String prenom;  
    int points;  
}
```

Chez DanielCraft, Lea modelise un **Produit** (nom, prix). Max confond encore classe et fichier. Sam dit : "le plan / la chose".

★ A RETENIR

Classe = plan. Objet = instance du plan.

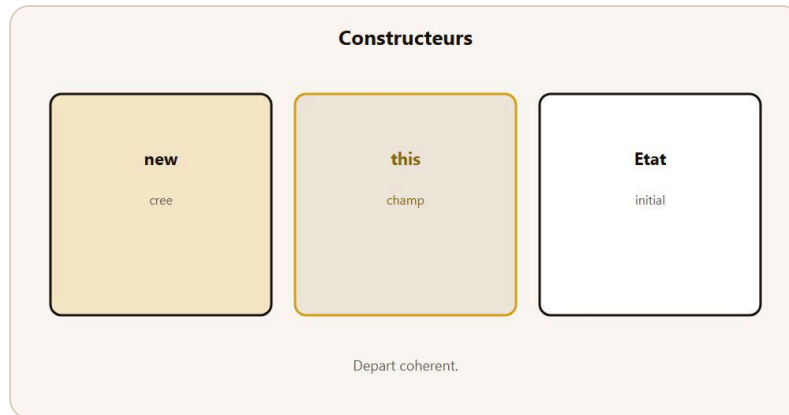
A toi

Ecris une classe **Livre** avec titre et pages.

♦ ASTUCE

Un concept metier = souvent une classe.

Chapitre 11 - Constructeurs



Constructeur : préparer l'objet à la création.

Le **constructeur** prépare l'objet à la création.

```
public Client(String prenom) {  
    this.prenom = prenom;  
    this.points = 0;  
}
```

`new Client("Max")` appelle le constructeur. Chez DanielCraft, Lea initialise proprement. Max laisse des champs à null et trébuche plus tard.

★ A RETENIR

Constructeur = état de départ cohérent.

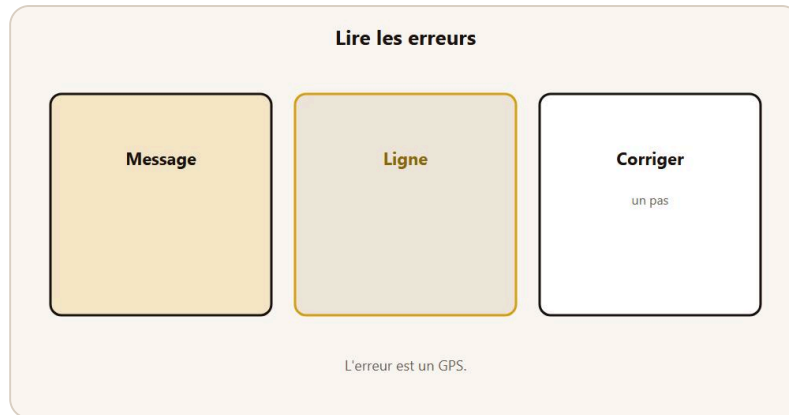
A toi

Constructeur `Produit(String nom, double prix)`.

⚠ ATTENTION

Oublier d'assigner un champ : valeurs par défaut / null.

Chapitre 12 - Lire les erreurs



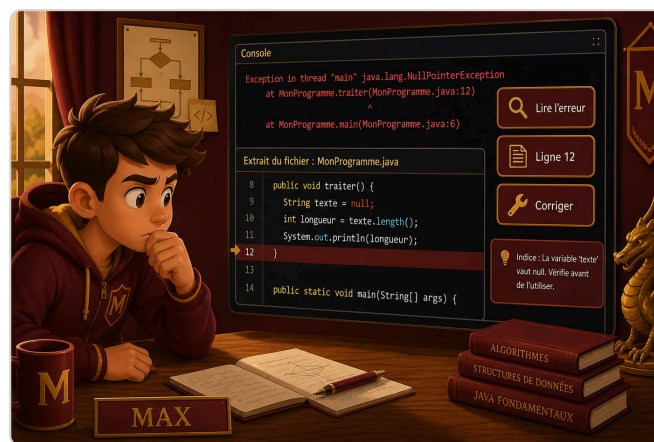
Lire une erreur : message, ligne, corriger.

Le compilateur et la JVM parlent. Lis le **fichier**, la **ligne**, le **message**. Chez DanielCraft, Max respire avant de tout reecrire. Lea corrige une typo. Sam enseigne : "l'erreur est un GPS, pas une insulte."

Exceptions frequentes debutant : `NullPointerException`, index hors tableau, type incompatible.

★ A RETENIR

Message + ligne + dernier changement = methode de debug.



Exemple : Max lit la ligne avant de paniquer.

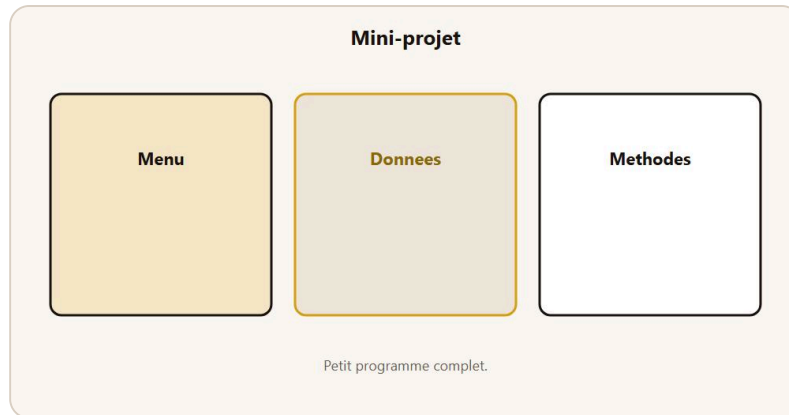
A toi

Casse volontairement un programme et note les 3 infos de l'erreur.

♦ ASTUCE

Une seule modification entre deux essais.

Chapitre 13 - Mini-projet



Mini-projet : un petit programme qui tient debout.

Programme console : **liste de tâches** ou **compteur de points**.

1. 1. Classe ou tableaux pour stocker
2. 2. Menu texte (ajouter / afficher / quitter)
3. 3. Boucle tant que pas quitter
4. 4. Au moins une methode separee
5. 5. Gerer une saisie invalide simplement

Lea fait le menu. Max gere le tableau. Sam teste les cas limites.

★ A RETENIR

Petit programme complet > dix bouts isoless.



Exemple : Lea, Max et Sam valident le mini-projet.

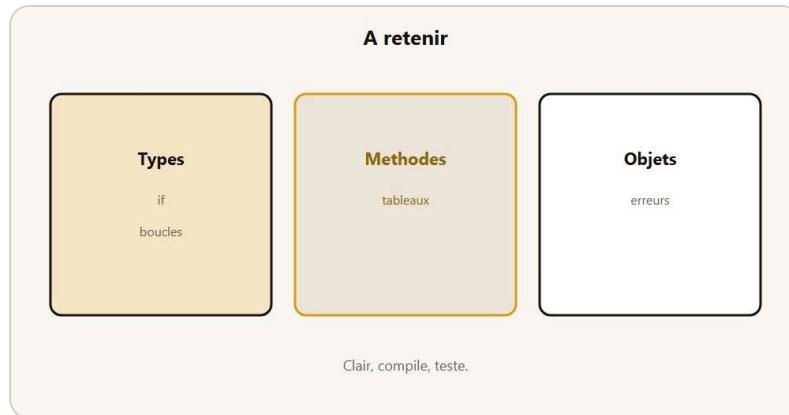
A toi

Une page "Mini-projet Java - date" avec les 5 points coches.

 ATTENTION

Sans menu de sortie, tu restes bloqué dans la boucle.

Chapitre 14 - Retenir l'essentiel



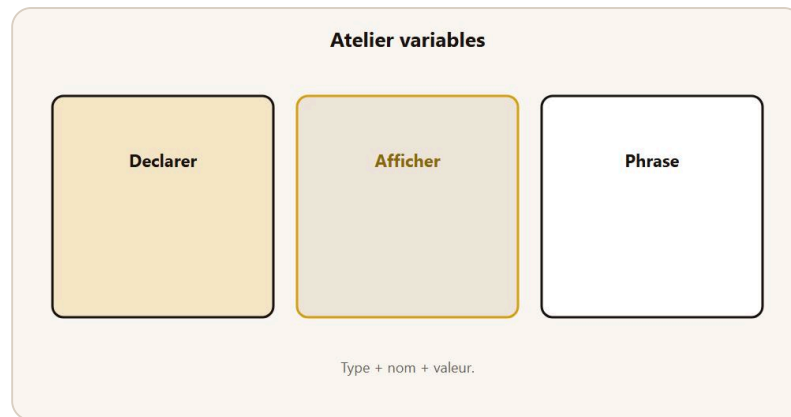
Variables, methodes, objets : la carte.

Java compile puis execute (JVM). Variables typees. if / boucles. Methodes. Tableaux index 0. Classes / objets / constructeurs. Lire les erreurs calmement.

★ A RETENIR

Clarte, petits pas, tester souvent.

Chapitre 15 - Atelier variables



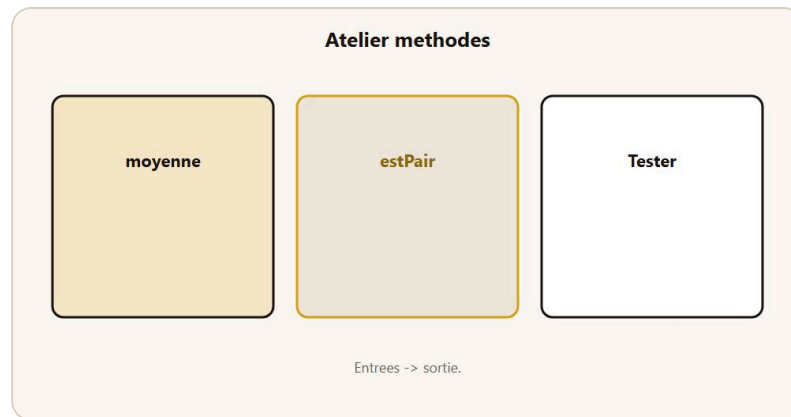
Atelier : declarer et afficher.

Declare nom, age, taille, majeur. Affiche une phrase complete.

★ A RETENIR

Declarer, initialiser, afficher.

Chapitre 16 - Atelier methodes



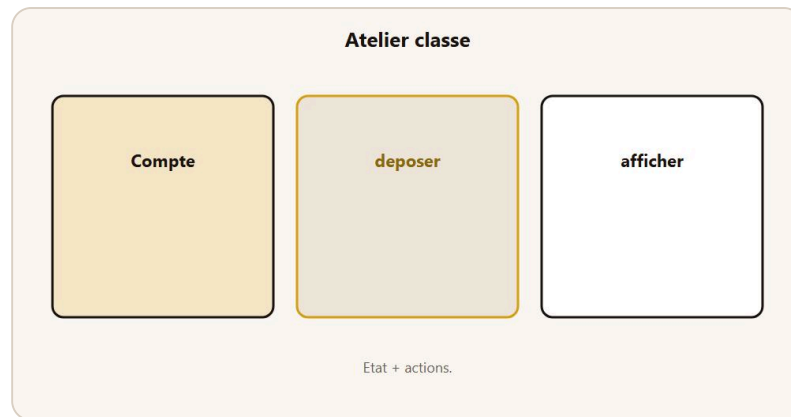
Atelier : decouper en methodes.

Ecris `moyenne(double a, double b)` et `estPair(int n)`.

★ A RETENIR

Entrees -> traitement -> sortie.

Chapitre 17 - Atelier classe



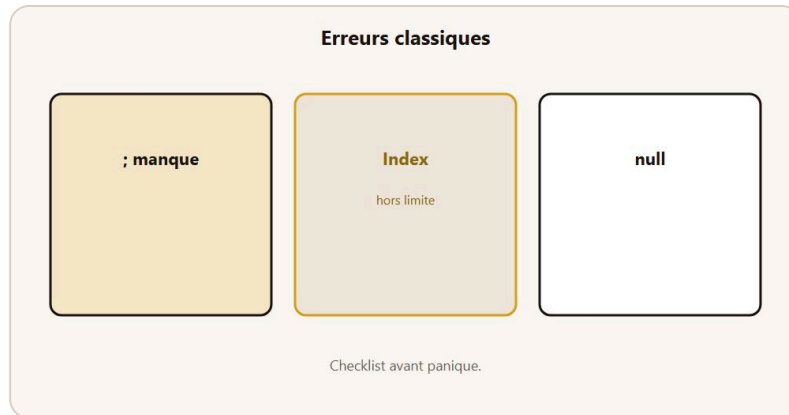
Atelier : une petite classe metier.

Classe `Compte` avec `solde`, constructeur, `deposer`, `afficher`.

★ A RETENIR

Etat + actions = objet utile.

Chapitre 18 - Erreurs classiques



Pieges : ; oublie, types, NullPointerException.

Point-virgule manque. `{` non ferme. `==` vs `=`. Index hors limites. Null. Confondre Java et JS. Methode non statique appelee maladroitement depuis main au debut.

★ A RETENIR

Les pieges sont repetables : checklist.

Chapitre 19 - Bonnes pratiques



Pratiques : noms clairs, petits pas, tester.

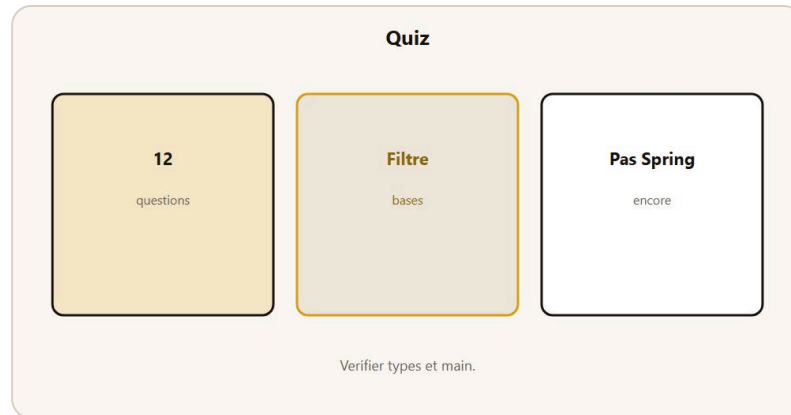
1. Noms clairs
2. Une idee par methode
3. Tester apres chaque brique
4. Lire les erreurs
5. Indentation
6. Commentaires utiles, pas de roman
7. Pas de copier-coller x10
8. Petits programmes complets
9. Dossier range
10. Relire demain

★ A RETENIR

Propre et testable bat impressionnant et casse.

QUIZ

Quiz final



Quiz Java bases.

Question 1

Java s'exécute souvent via :

- A) Uniquement Excel
- B) La JVM
- C) Uniquement le navigateur comme HTML

Question 2

Le point d'entrée classique est :

- A) main
- B) CSS
- C) DROP TABLE

Question 3

Une variable Java a surtout :

- A) Un type et un nom
- B) Aucun type
- C) Uniquement une couleur

Question 4

if sert à :

- A) Répéter sans fin
- B) Choisir un chemin
- C) Compiler

Question 5

for sert à :

- A) Afficher une page web

- B) Repeter
- C) Creer une base

Question 6

Une methode :

- A) Ne peut jamais renvoyer de valeur
- B) Regroupe du code appellable
- C) Remplace le JDK

Question 7

L'index d'un tableau commence a :

- A) 1 toujours
- B) 0
- C) -1

Question 8

Une classe est :

- A) Un plan
- B) Uniquement un fichier image
- C) Un antivirus

Question 9

new AppelConstructeur :

- A) Cree un objet
- B) Supprime le disque
- C) Compile le HTML

Question 10

Face a une erreur :

- A) On ignore la ligne
- B) On lit message et ligne
- C) On change de langage aussitot

Question 11

String sert surtout a :

- A) Du texte
- B) Uniquement des entiers
- C) Des boucles

Question 12

Bonne pratique :

- A) Tout dans main sans noms
- B) Petits pas testes
- C) Copier 10 fois le meme bloc

Corrige

1B 2A 3A 4B 5B 6B 7B 8A 9A 10B 11A 12B

★ A RETENIR

JVM, types, methodes, objets, lire les erreurs.

Bravo



Bravo. Tu ecris du Java sans magie.

Tu as termine **Java - Les bases**.

Tu ecris, compiles, lances, structures avec methodes et objets simples. Chez DanielCraft, c'est le Niveau Base du futur pack Java.

Suite

Java intermediaire (collections, fichiers, exceptions fine). Ou croiser avec SQL. Ou un mini projet console plus riche.

★ A RETENIR

Clair, compile, teste.

♦ ASTUCE

Relis erreurs et methodes avant le prochain exo.

Tu as les briques. Maintenant, interroge.